Содержание

Введение ... 3
1. Обзор предметной области ... 5
1.1. Общая структура компилятора ... 5
1.2. Формальные грамматики ... 6
1.3. Методы синтаксического анализа ... 8
1.4. Обработка ошибок в компиляторах ... 11
1.5. Сравнение методов анализа ... 13
2. Конструкторский раздел ... 14
2.1. Архитектура учебной библиотеки parser_edsl ... 14
2.2. Архитектурные решения для расширения библиотеки ... 15
2.3. Проектирование поддержки LL(1)-анализа ... 16
2.4. Проектирование анализа по алгоритму Эрли ... 17
2.5. Проектирование поддержки приоритетов и ассоциативности в LALR(1)... 19
2.6. Проектирование поддержки пользовательских лексических ошибок ... 20
3. Технологический раздел ... 21
3.1. Выбор средств реализации ... 21
3.2. Реализация LL(1)-синтаксического анализатора ... 21
3.3. Реализация синтаксического анализа по алгоритму Эрли ... 23
3.4. Реализация поддержки приоритетов и ассоциативности в LALR(1) ... 25
3.5. Реализация пользовательских лексических ошибок ... 26
4. Тестирование ... 28
4.1. Методика тестирования ... 28
4.2. Разбор выражений методом LL(1) ... 28
4.3. Тестирование синтаксического анализа по алгоритму Эрли ... 29
4.4. Тестирование механизмов приоритетов и ассоциативности в LALR(1)... 30
4.5. Объявление пользовательских лексических ошибок ... 31
Список использованной литературы ... 35

Введение

Компиляторы и интерпретаторы — основа современных средств разработки. Они преобразуют программы из текстового представления в исполнимый код, и от качества их реализации зависит эффективность всего процесса разработки.

Синтаксический анализ — один из центральных этапов компиляции [1]. Его задача — проверить, соответствует ли структура программы формальной грамматике языка, и построить внутреннее представление для дальнейшей обработки. Существует несколько подходов к синтаксическому анализу [1]:

- нисходящий анализ (LL) — от корня к листьям;
- восходящий анализ (LR, LALR) — от листьев к корню;
- универсальные алгоритмы (Эрли, CYK) — работают с любыми контекстно-свободными грамматиками, но менее эффективны.

На практике выбор метода определяется балансом между выразительностью грамматики и производительностью анализатора.

При изучении дисциплины «Конструирование компиляторов» используется учебная библиотека parser_edsl. Она предоставляет встроенный язык (EDSL) для описания грамматик на Python и реализует базовый LALR(1)-анализ. Однако текущая версия имеет ограничения:

- отсутствует поддержка LL(1)-анализа;
- не реализован алгоритм Эрли для произвольных грамматик;
- нет механизма задания приоритетов и ассоциативности операций.

Эти ограничения затрудняют изучение альтернативных методов анализа и не позволяют продемонстрировать практические приёмы разрешения конфликтов в грамматиках.

Цель работы — расширить библиотеку parser_edsl, добавив поддержку LL(1)-анализа, алгоритма Эрли, механизма приоритетов операций и улучшенной обработки ошибок.

Задачи:

1. Изучить теоретические основы методов LL(1), LALR(1) и алгоритма Эрли.
2. Проанализировать архитектуру исходной версии библиотеки.
3. Реализовать LL(1)-анализатор с построением таблицы разбора.
4. Реализовать анализатор по алгоритму Эрли.
5. Добавить поддержку приоритетов и ассоциативности для разрешения shift/reduce конфликтов в LALR(1).
6. Реализовать механизм пользовательских лексических ошибок с указанием позиции (строка, колонка) и описанием проблемы.
7. Протестировать реализованные расширения на примерах грамматик.

Объект исследования — методы синтаксического анализа контекстно-свободных грамматик.

Предмет исследования — алгоритмы LL(1), LALR(1) с разрешением конфликтов и алгоритм Эрли, а также их программная реализация.

Методы: анализ литературы, проектирование программной архитектуры, реализация на Python, экспериментальное тестирование.

Работа состоит из введения, четырёх разделов (обзор предметной области, конструкторский, технологический, экспериментальный), заключения и списка источников.

1. Обзор предметной области

1.1. Общая структура компилятора

Компилятор — это программа, которая переводит исходный код с языка высокого уровня в машинный код или промежуточное представление. В отличие от интерпретатора, компилятор обрабатывает всю программу целиком до её выполнения, что позволяет проводить глобальные оптимизации и выявлять ошибки заранее.

Классическая архитектура компилятора разделяется на две части: front-end (анализ) и back-end (синтез). Front-end отвечает за понимание исходного кода, back-end — за генерацию целевого кода. Такое разделение позволяет переиспользовать компоненты: один front-end может работать с несколькими back-end'ами для разных платформ (как в GCC или LLVM).

Лексический анализ (сканирование) — первый этап обработки. Лексер читает поток символов и группирует их в лексемы (токены). Каждая лексема имеет тип и, опционально, значение. Лексер отбрасывает пробельные символы и комментарии, обрабатывает лексические ошибки (незакрытые строки, недопустимые символы). Технически лексер реализуется как конечный автомат, построенный по регулярным выражениям.

Синтаксический анализ (парсинг) — центральный этап компиляции. Парсер проверяет, соответствует ли последовательность токенов грамматике языка, и строит дерево разбора. Существует два типа деревьев: дерево разбора (parse tree), точно отражающее структуру грамматики включая все промежуточные нетерминалы, и абстрактное синтаксическое дерево (AST), упрощённое представление, содержащее только семантически значимые узлы. Именно на этапе синтаксического анализа выявляется большинство структурных ошибок: пропущенные скобки, неправильный порядок конструкций, незакрытые блоки.

Семантический анализ дополняет синтаксический, проверяя ограничения, которые невозможно выразить в контекстно-свободной грамматике: соответствие типов в выражениях, объявление переменных перед использованием, корректность вызова функций (число и типы аргументов),

область видимости идентификаторов. На этом этапе строится таблица символов — структура данных, хранящая информацию обо всех объявленных идентификаторах.

Генерация промежуточного представления создаёт код на внутреннем языке компилятора. Типичные формы: трёхадресный код, SSA (Static Single Assignment), байт-код виртуальной машины. Промежуточное представление не зависит от целевой платформы, что упрощает оптимизацию и перенос на другие архитектуры.

Оптимизация улучшает код по различным критериям: скорость выполнения, размер, энергопотребление. Оптимизации делятся на машинно-независимые (удаление мёртвого кода, свёртка констант, вынос инвариантов из циклов) и машинно-зависимые (распределение регистров, планирование инструкций).

Генерация целевого кода — финальный этап, на котором промежуточное представление транслируется в машинный код или ассемблер целевой платформы.

В данной работе основное внимание уделяется синтаксическому анализу и связанным с ним механизмам обработки ошибок.

1.2. Формальные грамматики

Формальная грамматика — это математический аппарат для описания синтаксиса языка. Грамматика задаёт правила, по которым из элементарных символов можно построить допустимые конструкции языка.

По иерархии Хомского грамматики делятся на четыре типа:

- тип 0 (неограниченные) — эквивалентны машине Тьюринга;
- тип 1 (контекстно-зависимые) — левая часть правила не длиннее правой;
- тип 2 (контекстно-свободные) — левая часть содержит ровно один нетерминал;
- тип 3 (регулярные) — описываются регулярными выражениями.

Для описания синтаксиса языков программирования используются контекстно-свободные грамматики (КС-грамматики), поскольку они достаточно выразительны для описания вложенных конструкций (скобки, блоки, рекурсивные структуры), но при этом допускают эффективный разбор [2].

Формально КС-грамматика определяется четвёркой $G = (N, \Sigma, P, S)$, где N — конечное множество нетерминальных символов (синтаксические категории); Σ — конечное множество терминальных символов (лексемы языка), причём $N \cap \Sigma = \emptyset$; P — множество продукций (правил вывода) вида $A \to \alpha$, где $A \in N$, а $\alpha \in (N \cup \Sigma)^*$; $S \in N$ — начальный (стартовый) символ грамматики [2].

Вывод (деривация) — это последовательность применений продукций, преобразующая начальный символ в строку терминалов. Различают левосторонний вывод, при котором на каждом шаге заменяется самый левый нетерминал (используется в LL-анализаторах), и правосторонний вывод, при котором заменяется самый правый нетерминал (используется в LR-анализаторах в обратном порядке).

Неоднозначность грамматики возникает, когда для одной строки существует несколько различных деревьев разбора. Неоднозначность устраняется либо переписыванием грамматики (введением уровней приоритета через дополнительные нетерминалы), либо внешними механизмами — заданием приоритетов и ассоциативности операций.

Множества FIRST и FOLLOW — ключевые понятия для построения таблиц разбора [1]. $FIRST(\alpha)$ — множество терминалов, которыми может начинаться строка, выводимая из α. Если α выводит пустую строку ε, то $\varepsilon \in FIRST(\alpha)$. $FOLLOW(A)$ — множество терминалов, которые могут стоять непосредственно после нетерминала A в каком-либо выводе. Всегда включает маркер конца входа \$ для стартового символа.

Алгоритм вычисления FIRST: для терминала a $FIRST(a) = \{a\}$; для продукции $A \to \varepsilon$ добавить ε в $FIRST(A)$; для продукции $A \to X_1X_2...X_n$ добавить $FIRST(X_1)$ без ε в $FIRST(A)$, если $\varepsilon \in FIRST(X_1)$, добавить $FIRST(X_2)$ без ε и так далее.

Алгоритм вычисления FOLLOW: добавить $ в FOLLOW(S) для стартового символа; для продукции A → αBβ добавить FIRST(β) без ε в FOLLOW(B); если β выводит ε или β пусто, добавить FOLLOW(A) в FOLLOW(B).

1.3. Методы синтаксического анализа

Методы синтаксического анализа делятся на две большие группы: нисходящие (top-down) и восходящие (bottom-up).

Нисходящие анализаторы строят дерево разбора от корня (стартового символа) к листьям (терминалам). Они пытаются «предсказать», какое правило применить, глядя на следующий токен. К этому классу относятся LL-анализаторы и рекурсивный спуск.

Восходящие анализаторы работают в обратном направлении: от листьев к корню. Они накапливают токены на стеке и «сворачивают» их в нетерминалы, когда распознают правую часть правила. К этому классу относятся LR, SLR, LALR-анализаторы.

Основные отличия: направление построения дерева (от корня к листьям для нисходящих, от листьев к корню для восходящих), тип вывода (левосторонний для LL, правосторонний в обратном порядке для LR/LALR), обработка левой рекурсии (запрещена для LL, допускается для LALR), класс грамматик (более узкий для LL, более широкий для LALR), сложность реализации (проще для LL, сложнее для LALR).

1.3.1. LL(1)-анализ

LL(1) означает: L (Left-to-right) — чтение входа слева направо; L (Leftmost derivation) — построение левостороннего вывода; 1 — один символ предпросмотра.

LL(1)-анализатор использует таблицу разбора M[A, a], где A — нетерминал, a — терминал или маркер конца $. Ячейка содержит номер продукции, которую нужно применить, или признак ошибки [1].

Алгоритм построения таблицы: для каждой продукции $A \to \alpha$, для каждого терминала a из FIRST(α) записать $A \to \alpha$ в M[A, a]; если $\varepsilon \in$ FIRST(α), то для каждого $b \in$ FOLLOW(A) записать $A \to \alpha$ в M[A, b].

Алгоритм разбора использует стек, инициализированный стартовым символом и маркером конца. На каждой итерации: если вершина стека X — терминал, сравнить с текущим токеном a; при совпадении снять X со стека и прочитать следующий токен, иначе ошибка. Если X — нетерминал, найти продукцию в M[X, a]; если она есть, снять X и положить правую часть продукции на стек в обратном порядке, иначе ошибка.

Ограничения LL(1): грамматика не является LL(1), если существует левая рекурсия $A \to A\alpha$ (решение — преобразование в правую рекурсию), есть левая факторизация $A \to \alpha\beta_1 \mid \alpha\beta_2$ (решение — вынесение общего префикса), или пересекаются множества FIRST для разных альтернатив одного нетерминала.

Несмотря на ограничения, LL(1) популярен благодаря простоте: его легко реализовать вручную методом рекурсивного спуска, а таблица разбора наглядна и удобна для отладки.

1.3.2. LALR(1)-анализ

LALR(1) — Look-Ahead LR(1) — наиболее практичный вариант LR-анализа. Он сочетает мощность LR(1) с компактностью таблиц SLR [1].

LR-анализаторы основаны на конечном автомате, состояния которого представляют «ситуации» разбора. Каждая ситуация (LR-пункт) — это продукция с отмеченной позицией: $A \to \alpha \bullet \beta$. Точка показывает, какая часть правила уже распознана (α), а какая ещё ожидается (β).

Таблица LALR(1) состоит из двух частей: ACTION[s, a] — действие в состоянии s при терминале a (shift s' — перейти в состояние s' и прочитать токен; reduce $A \to \alpha$ — свернуть $|\alpha|$ символов в A; accept — разбор завершён успешно; error — синтаксическая ошибка); GOTO[s, A] — состояние перехода после свёртки к нетерминалу A.

Алгоритм разбора: стек инициализируется состоянием 0; на каждой итерации берётся состояние s с вершины стека и текущий токен a, затем

выполняется ACTION[s, a]. При shift s' на стек кладутся токен a и состояние s', затем читается следующий токен. При reduce A → α со стека снимается 2|α| элементов (символы и состояния), берётся новое состояние s' с вершины стека, на стек кладутся A и GOTO[s', A], вызывается семантическое действие для A → α. При accept возвращается успех, при error обрабатывается ошибка.

Конфликты и их разрешение: shift/reduce конфликт возникает, когда неясно, сдвигать токен или сворачивать (появляется в неоднозначных грамматиках); reduce/reduce конфликт возникает, когда несколько правил могут быть применены (обычно указывает на ошибку в грамматике) [1].

Для разрешения shift/reduce конфликтов используются приоритеты и ассоциативность. Правила разрешения: сравниваются приоритеты — побеждает действие с бóльшим приоритетом; при равных приоритетах левая ассоциативность даёт reduce, правая — shift; NonAssoc запрещает цепочки.

1.3.3. Алгоритм Эрли

Алгоритм Эрли (Earley, 1970) — универсальный метод синтаксического анализа, работающий с любыми контекстно-свободными грамматиками, включая неоднозначные и леворекурсивные [3].

Вместо построения детерминированного автомата алгоритм Эрли поддерживает множество «гипотез» о возможных разборах. Для каждой позиции во входной строке строится множество элементов Эрли (Earley items).

Элемент Эрли — это тройка (A → α•β, i), где A → αβ — продукция грамматики, точка • показывает прогресс разбора, i — позиция во входе, где началось распознавание A.

Три операции алгоритма: Prediction (предсказание) — если точка стоит перед нетерминалом B, добавляются все продукции для B; Scanning (сканирование) — если точка стоит перед терминалом a и он совпадает с текущим токеном, точка продвигается; Completion (завершение) — если точка дошла до конца правила, находятся все элементы, ожидавшие этот нетерминал, и в них продвигается точка [3].

Сложность: худший случай $O(n^3)$ для произвольных грамматик, $O(n^2)$ для однозначных грамматик, $O(n)$ для LR-грамматик [3].

Преимущества: работает с любыми КС-грамматиками без преобразований, корректно обрабатывает левую рекурсию, может обнаруживать неоднозначность грамматики для конкретного входа.

Недостатки: медленнее LALR(1) на практике, сложнее интегрировать семантические действия, для неоднозначных грамматик нужна дополнительная логика выбора разбора.

Обнаружение неоднозначности. При работе алгоритма Эрли неоднозначность проявляется, когда для одного и того же элемента (prod_index, dot, origin) в chart-множестве обнаруживается альтернативный путь вывода — при Scanning или Completion элемент с тем же ключом уже присутствует в множестве. Это можно использовать для выдачи предупреждения или генерации ошибки.

Обнаружение ошибок. Алгоритм Эрли естественным образом обнаруживает синтаксические ошибки: если после обработки множества S(i) следующее множество S(i+1) оказывается пустым, значит текущий токен не ожидался. Если в последнем множестве S(n) отсутствует завершающий элемент, входная строка не принадлежит языку грамматики.

1.4. Обработка ошибок в компиляторах

Качественная обработка ошибок — один из главных факторов удобства компилятора. Программист проводит значительную часть времени, исправляя ошибки, и чем точнее компилятор укажет проблему, тем быстрее она будет устранена.

Ошибки классифицируются по этапам компиляции. Лексические ошибки: незакрытая строка, недопустимый символ, некорректное число. Синтаксические ошибки: пропущенная точка с запятой, лишняя скобка, неправильный порядок конструкций. Семантические ошибки: несоответствие типов, необъявленная переменная, повторное объявление.

Лексические ошибки. Лексер первым встречает текст программы и должен сообщать о проблемах максимально точно. Хорошее сообщение об ошибке содержит позицию (строка и колонка или смещение в байтах), контекст (фрагмент исходного кода с указателем на проблемное место) и описание (что именно не так и, возможно, как исправить).

В библиотеке parser_edsl для этого введён класс LexerUserError, позволяющий определять пользовательские ошибки с указанием позиции.

Синтаксические ошибки. Синтаксический анализатор обнаруживает ошибки, когда входная последовательность токенов не соответствует ни одному правилу грамматики. Существуют различные стратегии восстановления после ошибок.

Режим паники (panic mode) — пропускать токены до «синхронизирующего» символа (точка с запятой, закрывающая скобка). Простой метод, но теряет много контекста.

Восстановление на уровне фразы — заменить, вставить или удалить токен локально. Например, вставить пропущенную точку с запятой.

Продукции ошибок — добавить в грамматику правила для типичных ошибок.

Глобальная коррекция — найти минимальное редактирование, делающее программу корректной. Теоретически оптимально, но слишком дорого на практике.

Сообщения об ошибках: лучшие практики. Необходимо указывать ожидаемое и полученное, не сообщать о каскадных ошибках (одна пропущенная скобка может вызвать десятки последующих ошибок), по возможности предлагать исправление, использовать цветовое выделение в терминале для наглядности.

Современные компиляторы (Clang, Rust, Elm) уделяют огромное внимание качеству сообщений об ошибках, превращая их в обучающий инструмент для программиста.

Координаты в parser_edsl. Библиотека поддерживает передачу координат (позиций во входном тексте) в семантические действия через ExAction. Это

позволяет формировать информативные сообщения об ошибках на этапе семантического анализа, указывая точное место в исходном коде.

1.5. Сравнение методов анализа

Для выбора метода синтаксического анализа необходимо учитывать характеристики грамматики, требования к производительности и сложность реализации.

Сравнение основных характеристик методов. LL(1): класс грамматик ограниченный, левая рекурсия запрещена, сложность разбора O(n), размер таблицы маленький, обработка неоднозначности приводит к ошибке, простота отладки высокая, интеграция семантических действий простая. LALR(1): класс грамматик широкий, левая рекурсия разрешена, сложность разбора O(n), размер таблицы средний, обработка неоднозначности через конфликты, простота отладки средняя, интеграция семантических действий простая. Earley: класс грамматик любые КС, левая рекурсия разрешена, сложность разбора $O(n^3)$, таблица отсутствует, обработка неоднозначности через предупреждение или ошибку, простота отладки сложная, интеграция семантических действий сложнее.

Рекомендации по выбору: LL(1) подходит для простых DSL, конфигурационных файлов, учебных целей, его легко реализовать вручную. LALR(1) подходит для языков программирования, обеспечивает хороший баланс мощности и производительности, является стандартом индустрии (yacc, bison). Earley подходит для анализа естественного языка, произвольных грамматик, прототипирования, когда важнее универсальность, чем скорость.

2. Конструкторский раздел

2.1. Архитектура учебной библиотеки parser_edsl

Библиотека parser_edsl — это каркас для описания грамматик и выполнения синтаксического анализа. Главная идея — использовать встроенный язык (EDSL) на Python, чтобы описание грамматики выглядело максимально близко к математической записи [5].

Архитектура библиотеки построена по принципу разделения ответственности и включает четыре основных компонента:

1. Представление грамматических символов. Иерархия классов Symbol, BaseTerminal, Terminal, LiteralTerminal, NonTerminal обеспечивает типобезопасное описание грамматики. Терминалы инкапсулируют регулярные выражения и функции преобразования. Нетерминалы хранят продукции и семантические действия.

2. Лексический анализ. Класс Lexer выполняет токенизацию входного текста по принципу "maximal munch" (longest match). При конфликтах учитывается приоритет терминалов. Поддерживается отслеживание позиций (строка, колонка) для диагностики ошибок.

3. Синтаксический анализ. Класс Parser — центральная точка интеграции различных методов разбора. Он собирает грамматику из взаимосвязанных нетерминалов, вычисляет FIRST-множества, делегирует построение таблиц и парсеров конкретным реализациям: ParsingTable для LALR(1), LL1Table для LL(1), EarleyParser для Earley.

4. Обработка ошибок. Иерархия исключений Error, ParseError, NonAssocError, LL1Error, EarleyAmbiguityError, LexerError, LexerUserError обеспечивает детализированную диагностику на всех этапах анализа.

Ключевое архитектурное решение — использование перегрузки оператора |= для добавления правил к нетерминалам. Это обеспечивает декларативный синтаксис описания грамматик, приближенный к форме Бэкуса-Наура. Внутренне оператор |= обрабатывает различные форматы входных данных: кортежи с лямбда-функциями, одиночные символы, пустые правила.

Строковые литералы автоматически оборачиваются в LiteralTerminal, а маркеры приоритетов извлекаются из списка символов и сохраняются отдельно.

Архитектура поддерживает принцип открытости/закрытости: библиотека открыта для расширения (добавление новых методов анализа), но закрыта для модификации (существующий код не изменяется при добавлении нового функционала).

2.2. Архитектурные решения для расширения библиотеки

При расширении библиотеки parser_edsl нужно было сохранить совместимость с исходной архитектурой и не сломать уже существующий код. Библиотека используется в учебных целях, поэтому её структура должна оставаться понятной.

Архитектурное решение 1: Паттерн "Стратегия" для методов разбора.

Вместо создания отдельных классов парсеров для каждого метода, был выбран подход с единой точкой входа (класс Parser) и делегированием конкретным реализациям. Это позволяет использовать единое API для всех методов анализа, переключать метод без изменения пользовательского кода и изолировать изменения в отдельных компонентах. В классе Parser добавлено поле method, которое определяет используемый алгоритм.

Архитектурное решение 2: Инкапсуляция таблиц разбора в отдельные классы.

Каждый метод анализа требует построения специфичных структур данных: таблицы ACTION/GOTO для LALR(1), таблицы M[A,a] для LL(1), множества элементов для Earley. Эти структуры инкапсулированы в отдельные классы: ParsingTable, LL1Table, EarleyParser. Такое разделение обеспечивает чёткое разграничение ответственности и возможность независимого тестирования каждого компонента.

Архитектурное решение 3: Единое представление грамматики.

Все методы анализа используют единую иерархию исключений с согласованной структурой: Error (абстрактный базовый класс), ParseError (общая ошибка разбора с позицией, неожиданным символом и списком ожидаемых),

NonAssocError (попытка цепочки неассоциативного оператора), EarleyAmbiguityError (неоднозначность грамматики), LL1Error (несовместимость грамматики с LL(1)), LexerError и LexerUserError (лексические ошибки). Это упрощает обработку ошибок в пользовательском коде независимо от выбранного метода анализа.

Архитектурное решение 4: Расширяемость через композицию.

Используется композиция вместо наследования: Parser содержит ссылки на реализации конкретных алгоритмов. Создаётся только тот объект, который соответствует выбранному методу, что исключает накладные расходы.

Архитектурное решение 5: Согласованный интерфейс обработки ошибок.

Все методы анализа используют единую иерархию исключений с согласованной структурой. Это упрощает обработку ошибок в пользовательском коде независимо от выбранного метода анализа.

2.3. Проектирование поддержки LL(1)-анализа

Добавление LL(1)-анализа потребовало реализации классического нисходящего синтаксического анализатора на основе таблицы разбора и стека.

Шаг 1: Определение структуры данных для таблицы LL(1).

Таблица разбора представляет собой отображение пары (нетерминал, терминал) в продукцию грамматики. Было решено использовать словарь Python с ключом-кортежем. Это обеспечивает константное время доступа при разборе и естественное представление отсутствующих записей.

Шаг 2: Проектирование алгоритма построения таблицы.

Алгоритм построения таблицы заполняет ячейки на основе множеств FIRST и FOLLOW. Ключевая проблема — обнаружение конфликтов. Если в одну ячейку таблицы пытаются записаться две разные продукции, грамматика не является LL(1). Было решено накапливать конфликты в отдельном списке для последующего анализа пользователем.

Шаг 3: Вычисление FOLLOW-множеств.

Множества FOLLOW вычисляются итеративно до достижения неподвижной точки. Использование итеративного алгоритма вместо

рекурсивного избегает проблем с циклическими зависимостями в сложных грамматиках.

Шаг 4: Проектирование алгоритма разбора.

LL(1)-разбор требует двух стеков: стек символов для управления процессом разбора и стек семантических данных для вычисления атрибутов. Основной цикл разбора извлекает символ с вершины стека и либо сопоставляет его с токеном, либо разворачивает нетерминал по правилу из таблицы.

Шаг 5: Обработка семантических действий.

Семантические действия выполняются после того, как собраны все дочерние элементы. Специальная функция проверяет готовность узла (набрано ли нужное количество детей) и вычисляет результат, передавая его на уровень выше. Это обеспечивает совместимость с семантикой LALR(1) и Earley.

2.4. Проектирование анализа по алгоритму Эрли

Алгоритм Эрли — универсальный метод синтаксического анализа для произвольных контекстно-свободных грамматик. Основная задача при проектировании — корректно представить состояния и организовать процесс анализа входной строки.

Шаг 1: Проектирование структуры данных для элемента Эрли.

Элемент Эрли (Earley item) — это состояние разбора, содержащее информацию о продукции, позиции точки в ней и месте начала разбора. Было решено использовать структуру с четырьмя полями: индекс продукции, позиция точки, начальная позиция (origin) и список обратных ссылок (back pointers) для восстановления дерева разбора.

Шаг 2: Проектирование структуры множеств состояний (chart).

Chart — это массив множеств состояний. Для n токенов создаётся n+1 множество. Используется структура данных, обеспечивающая быстрый поиск элементов и исключение дубликатов (словарь или хеш-таблица).

Шаг 3: Организация основного цикла обработки.

Алгоритм обрабатывает множества состояний последовательно. Внутри каждого множества используется очередь ("agenda") для элементов, ожидающих

обработки. Это гарантирует, что каждый элемент обрабатывается ровно один раз.

Шаг 4: Реализация операций алгоритма.

Реализованы три классические операции: Prediction (предсказание новых правил), Scanning (сопоставление терминалов) и Completion (завершение разбора нетерминала). Обратные ссылки сохраняются при Scanning и Completion для последующего построения дерева.

Шаг 5: Проектирование вычисления семантических действий.

После успешного разбора выполняется рекурсивный обход дерева разбора по обратным ссылкам (evaluate). Метод собирает атрибуты дочерних узлов и применяет к ним семантические функции, фильтруя пустые значения (None).

Шаг 6: Проектирование обработки ошибок.

Алгоритм Эрли должен корректно диагностировать синтаксические ошибки. Были спроектированы два механизма обнаружения ошибок. Первый — опустошение множества состояний: если после обработки множества S(i) следующее множество S(i+1) оказывается пустым, это означает, что ни один терминал не был просканирован, и текущий токен не ожидался. В этом случае поднимается ParseError с указанием неожиданного токена и списка ожидаемых терминалов, собранных из активных элементов S(i). Второй — отсутствие завершающего состояния: если в последнем множестве S(n) не найден элемент ($accept → S •, 0), значит разбор не завершён. Это может означать как неожиданный конец входа, так и наличие лишних символов. Поднимается ParseError с unexpected=EOF и списком ожидаемых терминалов.

Шаг 7: Проектирование обнаружения неоднозначности.

Неоднозначность грамматики проявляется, когда для одного и того же элемента (prod_index, dot, origin) в chart-множестве обнаруживается альтернативный путь вывода — при Scanning или Completion элемент с тем же ключом уже присутствует в множестве. Было спроектировано два режима поведения: режим предупреждения (по умолчанию) — выдача warnings.warn и возврат одного из результатов, и режим ошибки — поднятие

EarleyAmbiguityError. Выбор режима задаётся параметром ambiguity конструктора метода EARLEY.

2.5. Проектирование поддержки приоритетов и ассоциативности в LALR(1)

Расширение поддержки LALR(1)-анализа механизмами приоритетов позволяет корректно анализировать выражения с неоднозначными операторами (как в арифметике) без существенного усложнения грамматики.

Шаг 1: Проектирование классов для приоритетов.

Были введены специальные типы данных для обозначения ассоциативности (Left, Right, NonAssoc) и приоритета. Использование неизменяемых объектов обеспечивает их безопасное использование в качестве маркеров внутри правил грамматики.

Шаг 2: Интеграция в описание грамматики.

Маркеры приоритетов указываются прямо в правилах. При разборе правил они извлекаются и сохраняются отдельно, не влияя на структуру продукций.

Шаг 3: Проектирование алгоритма разрешения конфликтов.

При возникновении конфликта shift/reduce алгоритм сравнивает приоритет текущего терминала (shift) и приоритет правила, готового к свёртке (reduce).

- если приоритет shift выше — выбирается перенос;
- если приоритет reduce выше — выбирается свёртка;
- при равенстве решение принимается на основе ассоциативности (левая — reduce, правая — shift);
- при NonAssoc в таблицу записывается маркер NonAssocConflict; при попытке использовать это действие при разборе поднимается NonAssocError с указанием оператора и позиции.

Шаг 4: Проектирование стратегий разрешения конфликтов по умолчанию.

Для случаев, когда пользователь не указал приоритеты, были спроектированы резервные стратегии (fallback). При неразрешённом конфликте shift/reduce без явных приоритетов предпочтение отдаётся shift — поведение, стандартное для yacc/bison. При конфликте reduce/reduce выбирается правило с наибольшей длиной правой части (то есть не ε-правило), что предотвращает зацикливание на бесконечных свёртках пустых продукций. При равной длине выбирается правило с меньшим индексом.

2.6. Проектирование поддержки пользовательских лексических ошибок

Для повышения качества диагностики ошибок была спроектирована система пользовательской валидации токенов.

Шаг 1: Иерархия исключений.

Введено специализированное исключение (LexerUserError), которое отделяет семантические ошибки в токенах (например, "число слишком велико") от структурных ошибок лексера (неизвестный символ).

Шаг 2: Знание о контексте.

Функции валидации, которые пишет пользователь, не знают о позиции в файле. За это отвечает лексер. При возникновении ошибки валидации лексер перехватывает её, обогащает информацией о текущей строке и колонке, и пробрасывает дальше. Это разделяет ответственность: логика проверки — у пользователя, логика позиционирования — у лексера.

3. Технологический раздел

3.1. Выбор средств реализации

Расширения библиотеки parser_edsl реализованы на Python 3. Выбор языка обусловлен необходимостью сохранения совместимости с основной кодовой базой библиотеки.

Python предоставляет удобные средства для реализации алгоритмов синтаксического анализа [5]:

- встроенные словари и множества обеспечивают высокую производительность при работе с таблицами разбора и множествами состояний;
- механизм match/case (введенный в Python 3.10) значительно упрощает код диспетчеризации и обработки узлов дерева;
- модуль dataclasses позволяет компактно описывать структуры данных, такие как элементы Эрли или узлы дерева разбора, с автоматической генерацией методов сравнения и хеширования.

Библиотека не использует внешних зависимостей, полагаясь только на стандартную библиотеку Python (re, collections, abc, warnings, dataclasses), что упрощает её встраивание и переносимость.

3.2. Реализация LL(1)-синтаксического анализатора

Реализация LL(1)-анализа включает в себя построение таблицы разбора и исполняющий механизм (рантайм).

Класс LL1Table отвечает за построение таблицы предиктивного анализа. Таблица хранится в виде словаря, где ключом является кортеж (NonTerminal, Terminal).

Листинг 1 — Реализация класса LL1Table

```
class LL1Table:
    def __init__(self, gr):
        self.grammar = gr
        self.terminals = gr.terminals + (EOF_SYMBOL,)
        self.nonterms = gr.nonterms[1:]
        self.table = {}
        self.conflicts = []
        self.__build_table()
```

Метод __build_table, приведенный в листинге 2, реализует заполнение таблицы на основе множеств FIRST и FOLLOW. Одной из ключевых особенностей реализации является детектирование конфликтов (когда в одну ячейку попадает несколько правил) и их сохранение для отчетности, вместо немедленного аварийного завершения.

Листинг 2 — Реализация метода __build_table()

```
def __build_table(self):
    # ... (инициализация)
    for prod_index, (nt, prod, fold, _) in
enumerate(gr.productions):
        # Вычисление FIRST для правой части правила
        first_alpha = gr.first_set(prod)

        for terminal in first_alpha:
            if terminal is None: continue # Skip epsilon
            # Проверка на conflict FIRST/FIRST
            if key in self.table:
                self.conflicts.append(f"FIRST/FIRST конфликт...")
            else:
                self.table[key] = (prod_index, prod)

        # Обработка epsilon-правил через FOLLOW
        if None in first_alpha:
            follow = self.__compute_follow(nt)
            for terminal in follow:
                # Проверка на conflict FIRST/FOLLOW
                # ...
```

Вычисление множеств FOLLOW (__compute_follow) реализовано итеративно для поддержки циклических грамматик, обновляя множества до тех пор, пока они не перестанут изменяться (достижение неподвижной точки).

Сам процесс разбора реализован в методе __parse_ll1. Он использует два стека: один для символов грамматики (stack), другой для накопления семантических атрибутов (semantic_stack). Фрагмент функции представлен в листинге 3.

Листинг 3 — Реализация метода __parse_ll1

```
def __parse_ll1(self, text):
    # ... инициализация лексера ...
    stack = [EOF_SYMBOL, start_nt]

    while len(stack) > 0:
        X = stack.pop()

        if isinstance(X, BaseTerminal):
```

```
            # Сопоставление терминала
            if self.__terminal_matches_ll1(X, cur):
                # Успешное сопоставление, сохранение атрибута
                root_result['children'].append(cur.attr)
                self.__try_reduce_ll1(semantic_stack,
root_result)
            else:
                raise ParseError(cur.pos, cur.type, [X])
        else:
            # Раскрытие нетерминала по таблице
            key = (X, cur.type)
            if key not in self.ll1_table.table:
                raise ParseError(...)

            prod_index, prod = self.ll1_table.table[key]
            # Занесение правила в стек в обратном порядке
            for symbol in reversed(prod):
                stack.append(symbol)

            # Подготовка семантического действия
            semantic_stack.append({
                'fold': fold,
                'children': [],
                'expected_count': len(prod)
                # ...
            })
```

3.3. Реализация синтаксического анализа по алгоритму Эрли

Для реализации алгоритма Эрли был создан класс EarleyParser. Центральной структурой данных является EarleyItem — состояние парсера. Данная структура приведена в листинге 4.

Листинг 4 — Реализация класса EarleyItem

```
@dataclasses.dataclass
class EarleyItem:
    prod_index: int
    dot: int
    origin: int
    back_pointers: list = dataclasses.field(default_factory=list)

    def __hash__(self):
        # Хеширование только по значимым полям для корректной
работы set/dict
        return hash((self.prod_index, self.dot, self.origin))
```

Парсер управляет списком множеств состояний (chart), каждое из которых соответствует позиции во входном потоке. Обработка каждого множества (__process_chart) выполняется с использованием очереди (agenda) для поддержки динамического добавления элементов в процессе итерации.

Реализация основных операций алгоритма:

1. Prediction (Предсказание).
2. Completion (Завершение).

Обе операции представлены в листингах 5 и 6.

Листинг 5 — Реализация операции Prediction

```
def __predict(self, nonterminal, position):
    items = []
    for prod_index, (nt, prod, fold, _) in
enumerate(self.productions):
        if nt == nonterminal:
            # Создание нового item с точкой в начале
            items.append(EarleyItem(prod_index, 0, position))
    return items
```

Листинг 6 — Реализация операции Completion

```
def __complete(self, chart, completed_item, position):
    items = []
    completed_nt = self.productions[completed_item.prod_index][0]
    origin = completed_item.origin

    # Поиск элементов в родительском множестве, ожидающих этот
нетерминал
    for key, item in chart[origin].items():
        # ... проверка условия ...
        new_item = EarleyItem(item.prod_index, item.dot + 1,
item.origin)
        # Сохранение обратной ссылки для построения дерева
        new_item.back_pointers = item.back_pointers + [(origin,
position, completed_item)]
        items.append(new_item)
    return items
```

После завершения разбора метод __evaluate рекурсивно обходит сохраненные back_pointers, восстанавливая дерево разбора и применяя семантические действия к узлам, аналогично тому, как это делается в восходящем разборе.

Помимо основных операций, реализована обработка ошибок и обнаружение неоднозначности. После обработки каждого множества S(i) проверяется, не является ли следующее множество S(i+1) пустым. Если оно пусто, то ни один терминал не был просканирован на данном шаге, и поднимается ParseError с указанием неожиданного токена и списка ожидаемых терминалов. Для сбора ожидаемых терминалов реализован вспомогательный

метод __expected_terminals, обходящий активные элементы множества и извлекающий терминалы, стоящие после точки.

Если после обработки всех токенов в последнем множестве S(n) отсутствует завершающий элемент ($accept → S •, 0), поднимается ParseError с unexpected=EOF.

Обнаружение неоднозначности реализовано через отслеживание коллизий при Scanning и Completion: если при вставке нового элемента в chart-множество обнаруживается элемент с тем же ключом (prod_index, dot, origin), это означает наличие альтернативного пути вывода. В зависимости от параметра ambiguity конструктора, парсер либо выдаёт предупреждение (warnings.warn) и возвращает один результат, либо поднимает EarleyAmbiguityError. Данный параметр задаётся через конструктор метода: _EARLEY(ambiguity='warn') или _EARLEY(ambiguity='error').

3.4. Реализация поддержки приоритетов и ассоциативности в LALR(1)

Поддержка приоритетов реализована через метаданные, привязанные к правилам и терминалам. Для представления ассоциативности используются классы-маркеры, представленные в листинге 7.

Листинг 7 — Реализация классов-маркеров ассоциативности

```
@dataclasses.dataclass(frozen=True)
class Left: priority: int
@dataclasses.dataclass(frozen=True)
class Right: priority: int
@dataclasses.dataclass(frozen=True)
class NonAssoc: priority: int
```

При построении таблицы LALR(1), если обнаруживается конфликт shift/reduce, вызывается метод __resolve_conflict, представленный в листинге 8. Он реализует логику сравнения приоритетов, включая обработку NonAssoc и резервные стратегии для конфликтов без явных приоритетов.

Листинг 8 — Реализация алгоритма разрешения конфликтов shift/reduce

```
def __resolve_conflict(self, state_id, terminal, actions):
    shifts = [a for a in actions if isinstance(a, Shift)]
    reduces = [a for a in actions if isinstance(a, Reduce)]
```

```
    if len(shifts) == 1 and len(reduces) >= 1:
        # Получение приоритетов
        shift_prec_value =
self.__get_terminal_prec(terminal).priority
        reduce_prec_value = best_reduce_prec

        # Разрешение конфликта
        if shift_prec_value > reduce_prec_value:
            return [shifts[0]] # Побеждает Shift
        elif shift_prec_value < reduce_prec_value:
            return [best_reduce] # Побеждает Reduce
        else:
            # При равенстве решаем по ассоциативности
            if isinstance(best_reduce_prec, Left):
                return [best_reduce]
            elif isinstance(best_reduce_prec, Right):
                return [shifts[0]]
            # ...
    return actions
```

Класс NonAssocConflict выступает маркером в таблице разбора. При выполнении разбора, если в таблице встречается этот маркер вместо множества действий, поднимается исключение NonAssocError с указанием оператора и позиции.

Это позволяет парсеру автоматически выбирать правильное действие (например, свернуть 2+3 перед умножением в 2+3*4 или наоборот) на основе числовых приоритетов, заданных пользователем в грамматике. Резервные стратегии (fallback) обеспечивают работоспособность анализатора даже для грамматик без явных приоритетов: при shift/reduce без приоритетов предпочитается shift (как в yacc/bison), а при reduce/reduce выбирается правило с наибольшей длиной правой части, что предотвращает зацикливание на бесконечных свёртках пустых продукций.

3.5. Реализация пользовательских лексических ошибок

Механизм пользовательских ошибок реализован через специальное исключение LexerUserError. Это позволяет пользователю писать функции-валидаторы внутри определения терминалов. Пример такой функции приведен в листинге 9.

Листинг 9 — Пример реализации валидатора числа

```
def check_number(text):
    try:
        val = int(text)
        if val > 2147483647:
            # Пользователь выбрасывает ошибку без позиции
            raise LexerUserError("слошком большое число")
        return val
    except ValueError:
        raise LexerUserError(f"не число: {text}")

NUMBER = Terminal('NUMBER', r'\d+', check_number)
```

В методе Lexer.next_token, представленном в листинге 10, происходит перехват этой ошибки и обогащение её контекстной информацией.

Листинг 10 — Реализация метода next_token()

```
def next_token(self):
    # ... поиск токена ...
    try:
        attrib = domain.func(self.text[begin:end])
    except LexerUserError as e:
        # Лексер добавляет позицию (строка, колонка)
        e.pos = self.pos
        raise e
    # ...
```

Класс LexerUserError, представленный в листинге 11, спроектирован так, чтобы возвращать информативное сообщение, объединяющее текст пользователя и координаты ошибки.

Листинг 11 — Реализация класса LexerUserError

```
class LexerUserError(Exception):
    # ...
    @property
    def message(self):
        if self.pos:
            return f'{self.pos}: {self.user_message}'
        return self.user_message
```

Такой подход позволяет отделить логику проверки данных (бизнес-логику) от логики парсинга, сохраняя при этом информативность сообщений об ошибках.

4. Тестирование

4.1. Методика тестирования

Тестирование построено на сравнении результатов разбора с эталонными значениями. Для каждой функциональности создавалась тестовая грамматика, подавались строки с известным результатом.

4.2. Разбор выражений методом LL(1)

Для проверки реализации LL(1)-анализатора была использована грамматика арифметических выражений без левой рекурсии, представленная в листинге 12, допускающая предиктивный разбор. Грамматика построена с использованием вспомогательного нетерминала для устранения левой ассоциативности.

Листинг 12 — Грамматика арифметических выражений, совместимая с LL(1)

```
NUM = Terminal('NUM', r'[0-9]+', int)
E = NonTerminal('E')
T = NonTerminal('T')
Ep = NonTerminal("E'")
E |= (T, Ep, lambda t, ep: ep(t))
Ep |= ('+', T, Ep, lambda t, ep: lambda a: ep(a + t))
Ep |= (lambda: lambda a: a)
T |= NUM
p = Parser(E, method=LL1)
p.add_skipped_domain(r'\s+')
print(p.parse('1 + 2 + 3'))
```

Результат выполнения: 6, что соответствует ожидаемому значению

Дополнительно была проверена реакция анализатора на грамматику, не удовлетворяющую условиям LL(1), содержащую левую рекурсию.

При инициализации парсера была сгенерирована ошибка, свидетельствующая о наличии конфликта FIRST/FIRST. Грамматика представлена в листинге 13, а ошибка о наличии конфликта — в листинге 14.

Листинг 13 — Пример грамматики, не являющейся LL(1)

```
NUM = Terminal('NUM', r'[0-9]+', int)
E = NonTerminal('E')
E |= (E, '+', E, lambda a, b: a + b)
E |= NUM
Parser(E, method=LL1)
```

Листинг 14 — Диагностика конфликта FIRST/FIRST в LL(1)-анализе

```
Traceback (most recent call last):
  File "lab.py", line 21, in <module>
    Parser(E, method=LL1)
    ~~~~~~^^^^^^^^^^^^^^^^
  File "parser_edsl.py", line 1089, in __init__
    raise LL1Error(conflicts=self.ll1_table.conflicts)
parser_edsl.LL1Error: Грамматика не является LL(1): Конфликт
FIRST/FIRST для E на NUM: правила 1 и 2
```

4.3. Тестирование синтаксического анализа по алгоритму Эрли

Для тестирования реализации алгоритма Эрли была использована неоднозначная грамматика арифметических выражений с левой рекурсией и несколькими бинарными операторами, описанная в листинге 15.

Листинг 15 — Неоднозначная грамматика арифметических выражений

```
NUM = Terminal('NUM', r'[0-9]+', int)
E = NonTerminal('E')
E |= (E, '+', E, lambda a, b: a + b)
E |= (E, '*', E, lambda a, b: a * b)
E |= NUM
p = Parser(E, method=EARLEY)
p.add_skipped_domain(r'\s+')
print(p.parse('2 + 3 * 4'))
```

Результат выполнения: 20. При этом выдаётся предупреждение «Неоднозначность грамматики: существует несколько деревьев разбора» (UserWarning), поскольку для данного входа возможны два дерева разбора: (2+3)*4=20 и 2+(3*4)=14. Парсер по умолчанию работает в режиме ambiguity='warn' — выдаёт предупреждение и возвращает один из результатов.

Для проверки строгого режима обнаружения неоднозначности была использована та же грамматика с параметром ambiguity='error', как показано в листинге 16.

Листинг 16 — Грамматика в строгом режиме обнаружения неоднозначности

```
p = Parser(E, method=_EARLEY(ambiguity='error'))
p.add_skipped_domain(r'\s+')
p.parse('2 + 3 * 4')
```

В этом случае поднимается исключение EarleyAmbiguityError, сигнализирующее о том, что грамматика неоднозначна для данного входа.

Дополнительно была проверена обработка ошибок алгоритмом Эрли. При подаче неполного выражения (листинг 17) парсер корректно диагностирует ошибку.

Листинг 17 — Тестирование обработки ошибок в алгоритме Эрли

```
p = Parser(E, method=EARLEY)
p.add_skipped_domain(r'\s+')
p.parse('1 +')
```

Результат: ParseError «(1, 4): Неожиданный символ EOF, ожидалось NUM» — парсер обнаружил отсутствие завершающего состояния в последнем множестве и сформировал информативное сообщение об ошибке, указав позицию и список ожидаемых терминалов.

4.4. Тестирование механизмов приоритетов и ассоциативности в LALR(1)

Для проверки механизма разрешения конфликтов shift/reduce были использованы грамматики, представленные в листингах 18—20, с бинарными операторами, различающимися по ассоциативности.

Листинг 18 — Грамматика с левой ассоциативностью оператора вычитания

```
NUM = Terminal('NUM', r'[0-9]+', int)
E = NonTerminal('E')
E |= (E, '-', E, Left(1), lambda a, b: a - b)
E |= NUM
p = Parser(E, method=LALR1)
p.add_skipped_domain(r'\s+')
print(p.parse('10 - 5 - 2'))
```

Результат выполнения: 3, что соответствует выражению (10 - 5) - 2.

Листинг 19 — Грамматика с правой ассоциативностью оператора вычитания

```
NUM = Terminal('NUM', r'[0-9]+', int)
E = NonTerminal('E')
E |= (E, '-', E, Right(1), lambda a, b: a - b)
E |= NUM
p = Parser(E, method=LALR1)
p.add_skipped_domain(r'\s+')
print(p.parse('10 - 5 - 2'))
```

Результат выполнения: 7, что соответствует выражению 10 - (5 - 2).

Листинг 20 — Грамматика с неассоциативным оператором сравнения

```
NUM = Terminal('NUM', r'[0-9]+', int)
E = NonTerminal('E')
E |= (E, '<', E, NonAssoc(1), lambda a, b: a < b)
E |= NUM
p = Parser(E, method=LALR1)
p.add_skipped_domain(r'\s+')
print(p.parse('1 < 2'))
print(p.parse('1 < 2 < 3'))
```

Результат выполнения представлен в листинге 21.

Листинг 21 — Результат работы с неассоциативным оператором сравнения

```
True
Traceback (most recent call last):
  File "lab.py", line 60, in <module>
    print(p.parse('1 < 2 < 3'))
          ~~~~~~~^^^^^^^^^^^^^^
  File "parser_edsl.py", line 1163, in parse
    return self.__parse_lalr1(text)
           ~~~~~~~~~~~~~~~~~~^^^^^^
  File "parser_edsl.py", line 1186, in __parse_lalr1
    raise NonAssocError(pos=cur.pos.start,
operator=actions.operator)
parser_edsl.NonAssocError: (1, 7): оператор '<' не допускает
цепочки (NonAssoc)
```

Результаты подтверждают корректную работу механизма приоритетов и ассоциативности.

4.5. Объявление пользовательских лексических ошибок

Для проверки механизма пользовательских лексических ошибок был реализован терминал с пользовательской функцией валидации, приведенной в листинге 22, накладывающей ограничение на допустимый диапазон значений.

Листинг 22 — Пользовательская функция валидации числового литерала

```
def parse_limited(text):
    val = int(text)
    if val > 100:
        raise LexerUserError('Слишком большое число. Допустимы
только числа от 0 до 100.')
    return val
NUM = Terminal('NUM', r'[0-9]+', parse_limited)
E = NonTerminal('E')
E |= NUM
p = Parser(E, method=LALR1)
print(p.parse('999'))
```

При разборе строки 999 была получена ошибка, представленная в листинге 23.

Листинг 23 — Сообщение пользовательской лексической ошибки

```
Traceback (most recent call last):
  File "lab.py", line 72, in <module>
    print(p.parse('999'))
          ~~~~~~~^^^^^^^
  File "parser_edsl.py", line 1163, in parse
    return self.__parse_lalr1(text)
           ~~~~~~~~~~~~~~~~~~^^^^^^
  File "parser_edsl.py", line 1176, in __parse_lalr1
    cur = lexer.next_token()
  File "parser_edsl.py", line 1563, in next_token
```

```
    length, attr = d.match(self.text, offset)
                   ~~~~~~~^^^^^^^^^^^^^^^^^^^^
  File "parser_edsl.py", line 154, in match
    attrib = self.func(string[begin:end])
  File "lab.py", line 66, in parse_limited
    raise LexerUserError('Слишком большое число. Допустимы только
числа от 0 до 100.')
parser_edsl.LexerUserError: Слишком большое число. Допустимы
только числа от 0 до 100.
```

Заключение

В ходе выполнения работы была решена задача расширения функциональности учебной библиотеки синтаксического анализа parser_edsl. Цель работы достигнута — библиотека дополнена реализациями алгоритмов LL(1) и Эрли, а также механизмами управления приоритетами операций и обработки ошибок.

Основные результаты работы:

1. Проведен сравнительный анализ методов синтаксического анализа, обоснована необходимость поддержки различных классов грамматик в учебных целях.

2. Реализован нисходящий анализатор LL(1). Разработан алгоритм построения таблицы разбора на основе множеств FIRST и FOLLOW с автоматическим обнаружением конфликтов, что позволяет диагностировать несовместимость грамматики с методом LL(1).

3. Реализован универсальный алгоритм Эрли с обработкой ошибок и обнаружением неоднозначности. Это позволило обрабатывать произвольные контекстно-свободные грамматики, включая неоднозначные и имеющие левую рекурсию. При ошибках парсер диагностирует точную позицию и список ожидаемых терминалов (при опустошении множества состояний или отсутствии завершающего состояния). При неоднозначности — выдаёт предупреждение или поднимает EarleyAmbiguityError в зависимости от настройки.

4. Модифицирован LALR(1)-анализатор. Внедрена поддержка декларативного указания приоритетов и ассоциативности (Left, Right, NonAssoc, Prio), что обеспечивает автоматическое разрешение конфликтов типа «сдвиг-свёртка» без усложнения структуры грамматики. Для оператора NonAssoc реализован специализированный класс ошибки NonAssocError, диагностирующий попытку цепочечного использования неассоциативного оператора. Добавлены резервные стратегии разрешения конфликтов: при shift/reduce без приоритетов предпочитается shift (стандарт yacc/bison), при reduce/reduce выбирается правило с наибольшей длиной правой части, что предотвращает зацикливание.

5. Разработан механизм пользовательских ошибок. Добавлена возможность генерации исключений LexerUserError с точным указанием позиции (строка, колонка), что повышает информативность диагностики для конечного пользователя.

6. Выполнено тестирование. Создан набор тестов, подтверждающий корректность работы новых модулей и совпадение результатов разбора различными методами.

Практическая значимость полученных результатов заключается в создании комплексного инструмента для изучения теории компиляторов. Обновленная библиотека позволяет наглядно сравнивать эффективность и ограничения различных алгоритмов в рамках единого интерфейса. Это способствует более глубокому пониманию принципов работы синтаксических анализаторов и методов разрешения конфликтов в грамматиках.

Список использованной литературы

1. Ахо А. В., Сети Р., Ульман Дж. Д. Компиляторы: принципы, технологии и инструменты. – 2-е изд. – М.: Вильямс, 2008. – 1184 с.

2. Григоров А. Н. Формальные языки и методы трансляции. – М.: Издательство МГУ, 2004. – 256 с.

3. Earley J. An Efficient Context-Free Parsing Algorithm // Communications of the ACM. – 1970. – Vol. 13, No. 2. – P. 94–102.

4. Кормен Т. Х., Лейзерсон Ч. И., Ривест Р. Л., Штайн К. Алгоритмы: построение и анализ. – 3-е изд. – М.: Вильямс, 2013. – 1328 с.

5. Python Software Foundation. Python 3.14 Documentation [Электронный ресурс]. – Режим доступа: https://docs.python.org/ (дата обращения: 12.10.2025).